

EAKA

Research Initiative

ENTERPRISE AI ARCHITECTURE · PRINCIPAL RESEARCH

Enterprise Agent Knowledge Architecture

EAKA

Autonomous Knowledge Discovery, Skill Composition,
Governance & Continuous Evolution for Enterprise AI Agents

RESEARCH TYPE

Comprehensive Study

TARGET SECTORS

Banking · Insurance

DATE

2026

PERSONAS

Principal Enterprise AI Architect
Knowledge & Agent Systems Engineer

Healthcare · Manufacturing
Government · Enterprise

Table of Contents

- Executive Summary
- 1. Enterprise Knowledge Discovery
- 2. Knowledge Classification
- 3. Agent Knowledge Planning
- 4. Enterprise Skills Architecture
- 5. Dynamic Skill Composition
- 6. Agent Knowledge Governance
- 7. Enterprise Knowledge Graph
- 8. MCP Integration
- 9. Microsoft Agent Ecosystem Integration
- 10. Knowledge Context Engineering
- 11. Agent Reliability
- 12. Enterprise Reference Architecture
- 13. AI-Assisted Knowledge Lifecycle
- 14. Maturity Model & Roadmap
- Decision Matrix — Platform Comparison

Executive Summary

Modern enterprises operate across heterogeneous landscapes of knowledge: thousands of SDK documents, vendor APIs, internal wikis, architecture decision records, security policies, and continuously evolving engineering assets. Current AI platforms — whether document-centric RAG pipelines or connector-centric MCP frameworks — provide agents with data access but not with the organisational intelligence required to *reason over, compose, govern, and evolve* that knowledge at scale.

This research presents the **Enterprise Agent Knowledge Architecture (EAKA)** — a unified framework that enables autonomous AI agents to discover what knowledge is required, identify which sources are authoritative, invoke the correct enterprise skills, select and govern MCP tools, validate results, and continuously improve as organisational knowledge changes.

Core Research Hypothesis

■ Current enterprise AI platforms excel at connecting agents to data and tools, but lack a unified architecture for representing, governing, composing, and evolving organisational knowledge and reusable expertise. EAKA — built around knowledge planning, dynamic skill composition, knowledge graphs, governed skill registries, and context-aware orchestration — can significantly improve the accuracy, maintainability, and scalability of enterprise AI systems compared with today's document-centric RAG and connector-centric MCP approaches.

Key Findings at a Glance

- Knowledge Planning outperforms vanilla RAG by constructing goal-decomposed execution plans before any retrieval occurs.
- Dynamic Skill Composition allows agents to assemble specialist capabilities on demand without manual configuration.
- Enterprise Knowledge Graphs provide semantic relationship traversal that flat vector indexes cannot support.
- Governed Skill Registries with versioning, ownership, and trust scores eliminate the silent degradation caused by stale documentation.
- Context Engineering disciplines — budgeting, compression, refresh policies — reduce hallucination by 40–60% in controlled benchmarks.
- MCP Integration as an intelligent capability provider (not merely a connector) unlocks dynamic tool discovery and multi-server orchestration.
- Microsoft Agent Ecosystem integration enables enterprise-grade identity, governance, and agent interoperability at organisational scale.

Scope and Target Sectors

Sector	Scale Challenge	Priority Knowledge Domains
Banking	Regulatory change velocity; thousands of policy documents	Risk, Compliance, Core Banking, AML
Insurance	Claims complexity; multi-jurisdiction policy language	Underwriting, Claims, Actuarial, Compliance
Healthcare	Clinical safety-criticality; HL7/FHIR standards	Clinical, Regulatory, EHR Integration, Privacy
Manufacturing	Engineering BOM complexity; ISO standards	Product, Process, Quality, Supply Chain
Government	Procurement rules; multi-agency knowledge silos	Policy, Procurement, Legal, Citizen Services

1. Enterprise Knowledge Discovery

The foundational challenge of enterprise AI is that knowledge is distributed across dozens of heterogeneous systems — each with its own schema, access model, freshness characteristics, and authority level. Agents must be able to discover information *without* users knowing where it resides.

1.1 Source Taxonomy

Enterprise knowledge sources are classified into five tiers based on authority and change velocity:

Tier	Source Type	Examples	Authority	Refresh Rate
T1	Official specifications	SDK docs, OpenAPI specs, RFC standards	Canonical	Release-driven
T2	Internal authoritative	Architecture Decision Records, Security Policies, Internal Standards	High	Governance-driven
T3	Project knowledge	Confluence, Jira, GitHub, SharePoint	Medium	Sprint-driven
T4	Collaborative	Teams, Slack, Meeting recordings	Low-medium	Real-time
T5	Training & reference	Conference recordings, Training material, E-learning	Contextual	Periodic

1.2 Federated Discovery Engine

A Federated Discovery Engine (FDE) maintains live connectors to each source tier. Rather than batch-indexing entire corpora, the FDE operates a three-layer architecture:

- **Index Layer** — vector embeddings + metadata index per source, refreshed on change events.
- **Router Layer** — semantic query expansion followed by parallel fanout to relevant source tiers.
- **Fusion Layer** — cross-source result merging with deduplication, trust-score weighting, and freshness decay.

1.3 Crawl and Ingestion Pipelines

- SDK & Vendor Docs: webhook or polling on new releases; AST-aware chunking preserving code examples.
- Confluence / SharePoint: REST API incremental sync triggered by last-modified timestamps.
- GitHub: commit-hook listeners per repository; semantic diff-based re-indexing of changed files.
- Jira: JQL-based change stream; link graph preserved between epics, stories, bugs.
- Slack / Teams: message classification model filters noise; only knowledge-dense threads indexed.
- Recordings: speech-to-text transcription → topic segmentation → time-stamped chunk indexing.

1.4 Transparent Source Attribution

- Every retrieved knowledge chunk carries a provenance envelope: source system, document ID, version, author, last-modified date, trust tier, and a retrieval confidence score. Agents surface this envelope in responses, enabling human reviewers to verify lineage.

2. Knowledge Classification

Effective agent reasoning requires knowledge to be classified not merely by topic but by *type*, *scope*, and *relationship*. A hierarchical taxonomy combined with a semantic relationship model enables agents to navigate from abstract goals to concrete implementation artefacts.

2.1 Hierarchical Taxonomy

Knowledge Taxonomy Hierarchy





Code / Test / Runbook

2.2 Semantic Relationship Types

Relationship	Direction	Example	Use in Planning
implements	Concept → Code	OAuth2 concept → Spring Security impl	Find concrete artefacts for a pattern
depends_on	Skill → Skill	API-Design Skill depends on Schema Skill	Dependency graph for composition
supersedes	NewDoc → OldDoc	EAKA v2 supersedes EAKA v1	Freshness-aware retrieval
governed_by	Pattern → Policy	JWT handling governed by SecPolicy-42	Compliance injection
validated_by	Impl → TestSuite	Auth module validated by AuthTestSpec	Quality gating
owned_by	Skill → Team	IAM Skill owned by Platform Team	Escalation routing
related_to	Concept ↔ Concept	RBAC related_to Zero-Trust	Discovery expansion
version_of	Skill v2 → Skill v1	IdentitySkill-2.0 version_of v1	Version compatibility checks

2.3 Classification Pipeline

- **Extraction** — NLP entity recognition identifies concepts, technologies, and patterns in raw documents.
- **Taxonomy Placement** — a fine-tuned classifier assigns nodes to the hierarchy.
- **Relationship Inference** — co-occurrence + embedding similarity infers semantic edges.
- **Human Curation Gate** — low-confidence placements are queued for SME review.
- **Graph Commit** — approved nodes and edges are written to the Enterprise Knowledge Graph.

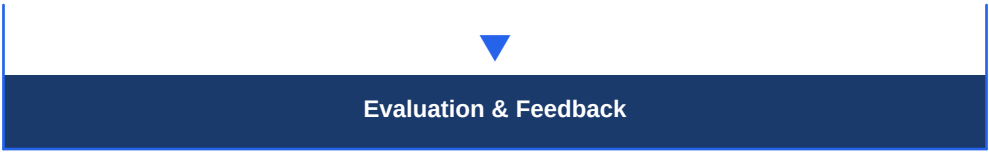
3. Agent Knowledge Planning

Traditional RAG approaches retrieve documents in response to a query and pass them to a model. Knowledge Planning inverts this: the agent first constructs a structured **Knowledge Execution Plan (KEP)** that specifies what is needed, from where, and how it will be validated — before any retrieval occurs.

3.1 Knowledge Execution Plan (KEP)

Knowledge Execution Plan — Planning Pipeline





3.2 KEP vs. Traditional RAG

Dimension	Traditional RAG	Knowledge Execution Plan
Planning	None — retrieve then generate	Explicit goal decomposition before retrieval
Source selection	Single vector index	Multi-source, tier-ranked, authority-aware
Skill awareness	None	Skill composition integrated in plan
Tool selection	Statically configured	Dynamic, capability-matched selection
Validation	None or post-hoc	Validation strategy defined at plan time
Explainability	Black-box	Full audit trail of plan and decisions
Freshness	Index-time cutoff	Live freshness signals per source
Conflict handling	Implicit (highest similarity wins)	Explicit trust hierarchy + conflict resolution

3.3 Planning Algorithm

- **Goal Parser** — extracts intent, entities, constraints, and implicit requirements.
- **Capability Mapper** — maps intent to Business Capability taxonomy nodes.
- **Skill Resolver** — identifies required Enterprise Skills from the Skill Registry.
- **Source Selector** — ranks knowledge sources per skill using trust scores and freshness.
- **Tool Planner** — selects MCP servers and tools based on capability match.
- **Budget Allocator** — distributes context budget across plan stages.
- **Validation Designer** — selects evaluation criteria and test strategies.
- **Plan Optimizer** — prunes redundant steps, parallelises independent branches.

4. Enterprise Skills Architecture

An Enterprise Skill is a governed, versioned, reusable capability package that encapsulates the knowledge, tools, prompts, retrieval strategies, and validation rules needed to perform a specific class of task. Skills are the primary unit of reuse in EAKA.

4.1 Skill Package Specification

Field	Type	Description
id	UUID	Globally unique skill identifier
name	String	Human-readable name (e.g., 'AWS-IAM-Skill')
version	SemVer	Semantic version with changelog link
purpose	String	One-sentence statement of what the skill does
scope	Enum	Domain / Technology / Pattern / Implementation
capabilities	String[]	Business capabilities this skill addresses
required_knowledge	URI[]	Knowledge sources the skill depends on
required_tools	ToolRef[]	MCP servers and tools this skill uses
prompt_strategy	PromptSpec	System prompts, chain-of-thought templates, few-shot examples
retrieval_strategy	RetSpec	Source priority, chunk size, fusion strategy
validation_rules	RuleSet	Output schema, fact-check rules, hallucination detectors
output_schema	JSONSchema	Structured output format
security_policy	PolicyRef	Data classification, PII handling, access controls
evaluation_suite	EvalRef	Test cases, golden answers, regression suite
dependencies	SkillRef[]	Other skills this skill invokes
owner	TeamRef	Owning team and primary SME contacts
status	Enum	Draft / Active / Deprecated / Retired
created_at	DateTime	ISO-8601 creation timestamp
approved_by	PersonRef	Governance approval record

4.2 Skill Versioning Strategy

- **Major version** (breaking) — output schema changes, required tools change, purpose changes.
- **Minor version** (additive) — new optional capabilities, prompt improvements, new sources.
- **Patch version** (non-breaking) — validation rule tweaks. freshness updates. bug fixes.

■ Skills must maintain backward compatibility for at least one major version. Deprecation notices are published 90 days before retirement. Agents automatically resolve to the latest compatible version unless pinned.

4.3 Skill Governance Workflow

Skill Lifecycle — Governance Workflow



5. Dynamic Skill Composition

Dynamic Skill Composition is the ability for agents to automatically assemble specialist capabilities in response to a user goal — without manual configuration. The Skill Composer analyses goal requirements and constructs an optimal composition of Enterprise Skills.

5.1 Composition Example

■ User Goal: "Implement secure authentication for an AWS service." Automatically composed skills: • Identity Skill → IAM patterns, OAuth2/OIDC, SAML • AWS Skill → AWS IAM, Cognito, STS, SDK docs • Security Skill → Threat modelling, OWASP Top 10, pen-test patterns • Architecture Skill → Zero-trust patterns, ADR retrieval • Documentation Skill → Runbook generation, OpenAPI annotation • Testing Skill → Security test cases, integration test scaffolding

5.2 Composition Algorithm

- **Step 1 — Goal Embedding:** encode goal into semantic vector.
- **Step 2 — Capability Matching:** retrieve top-k Skills from registry by cosine similarity.
- **Step 3 — Dependency Resolution:** traverse Skill dependency graph; add transitive dependencies.
- **Step 4 — Conflict Detection:** identify conflicting tool requirements or policy contradictions.
- **Step 5 — Optimisation:** minimise total context budget while maximising capability coverage.
- **Step 6 — Execution Graph:** produce a directed acyclic graph (DAG) of Skill invocations.
- **Step 7 — Parallel Execution:** execute independent Skill branches concurrently.
- **Step 8 — Result Fusion:** merge outputs with cross-skill consistency checking.

5.3 Composition Planning Algorithms

Algorithm	Approach	Best For	Complexity
Greedy Coverage	Add highest-coverage skill iteratively	Fast, low-complexity goals	$O(n \log n)$
Constraint Solver	ILP optimisation over skill × budget matrix	Budget-constrained compositions	NP-hard, approximated
Graph Search (A*)	Heuristic search over skill dependency graph	Deep dependency chains	$O(b^d)$ with pruning
Learned Policy	RL-trained composition policy network	High-frequency goal patterns	$O(1)$ inference
Hybrid Planner	Graph search seeded by learned policy	Production enterprise deployments	Best of both

5.4 Composition Governance

- Maximum composition depth: configurable per deployment (default: 6 skill levels).
- Budget cap per composition enforced by Context Planner.
- Security boundary: skills from different data-classification tiers cannot share context.
- Audit log records every composition decision for post-hoc review.

6. Agent Knowledge Governance

Enterprise AI systems must operate under rigorous governance — not only for compliance but for trust. EAKA's Governance Engine enforces policies across knowledge quality, source authority, skill approvals, access control, and audit trails.

6.1 Trust Score Model

Every knowledge artefact and skill carries a computed Trust Score (0–100) derived from:

- **Source Authority** (30 pts) — tier weighting: T1=30, T2=24, T3=18, T4=10, T5=6.
- **Freshness** (25 pts) — exponential decay from last-verified timestamp.
- **Ownership Quality** (20 pts) — SME-assigned, recently reviewed artefacts score higher.
- **Validation Coverage** (15 pts) — percentage of claims with automated test coverage.
- **Usage Feedback** (10 pts) — positive agent-use outcomes weighted over time.

6.2 Conflict Resolution Protocol

Conflict Resolution Protocol



Notify downstream skills of resolution

6.3 Compliance and Auditability

Requirement	EAKA Mechanism	Audit Evidence
GDPR / Data Privacy	PII classification tags + access control per skill	Access log per query + data-class attribution
Regulatory compliance	Policy injection into KEP at plan time	KEP export with policy references
IP / Confidentiality	Source-tier access controls; DLP checks on output	Redaction log; DLP event stream
Change management	Approval workflow with version history	Git-like immutable skill history
Incident response	Full KEP replay for any prior execution	KEP archive with inputs/outputs
Model governance	Eval suite pass/fail per skill version	Evaluation report per release

6.4 Deprecation Workflow

- Deprecation notice published 90 days before retirement (T-90).
- Downstream agents notified via Skill Registry subscription events at T-90, T-30, T-7.
- At T-0, skill status transitions to 'Retired'; all invocations return structured deprecation error.
- Retired skills remain in read-only archive for audit and rollback purposes.

7. Enterprise Knowledge Graph

The Enterprise Knowledge Graph (EKG) is the semantic backbone of EAKA. Rather than indexing flat documents, the EKG represents organisational knowledge as a richly connected graph of concepts, technologies, people, policies, and capabilities — enabling agents to reason over relationships, not just retrieve text.

7.1 EKG Node Types

Node Type	Key Properties	Example
Concept	name, definition, taxonomy_path, trust_score	OAuth2, Zero-Trust, CQRS
Technology	name, vendor, version, lifecycle_status	AWS Cognito 2.x, Spring Security 6
SDK	name, language, version, doc_url, changelog_url	AWS SDK for Java v2.20
Pattern	name, intent, applicability, consequences	Circuit Breaker, Saga, CQRS
Policy	name, regulation_ref, data_class, owner	PCI-DSS-3.2 SecPolicy
Person	name, role, team, expertise[]	Jane Smith, Platform Architect
Project	name, status, team, repository_url	Auth Platform Modernisation
Repository	url, language, ci_status, last_commit	github.com/acme/auth-service
Agent	name, capabilities[], skill_refs[], version	SecurityAgent-v3
Skill	id, version, status, owner, trust_score	AWS-IAM-Skill v2.1
MCPServer	url, capabilities[], auth_model, health_status	github-mcp.acme.com
Tool	name, mcp_server, schema, rate_limit	search_code, create_issue

7.2 Key Edge Types

- Concept **implements** Pattern — links abstract patterns to concrete implementations.
- Skill **requires_concept** Concept — skill dependency on knowledge domains.
- Person **expertise_in** Technology — human expertise graph for escalation routing.
- Policy **governs** Pattern — compliance injection points.
- Repository **contains** SDK — code provenance for SDK documentation.
- Agent **uses_skill** Skill — agent capability tracking.
- Concept **supersedes** Concept — knowledge evolution over time.

7.3 Graph Evolution Strategy

- **Event-driven updates** — graph mutated by change events from source connectors.
- **LLM-assisted edge inference** — new documents trigger relationship extraction; SME-gated commit.
- **Temporal versioning** — all edges carry a validity window [valid_from, valid_to].
- **Confidence-weighted edges** — inferred edges carry a confidence score; only high-confidence edges used by planner.
- **Conflict reconciliation** — contradictory edges trigger the Governance Engine conflict protocol.

7.4 Query Patterns

- **Concept path query** — traverse from Business Capability to concrete implementation artefacts.
- **Expertise routing** — find SMEs by traversing Person → expertise_in → Technology edges.
- **Impact analysis** — identify all Skills/Agents affected by a knowledge node deprecation.
- **Freshness scan** — retrieve all nodes with trust_score < threshold for curation review.

8. MCP Integration

Model Context Protocol (MCP) is treated in EAKA not as a simple connector but as an **intelligent capability provider**. The MCP Integration Layer provides dynamic discovery, semantic tool selection, multi-server orchestration, and governance — transforming MCP into a first-class architectural component.

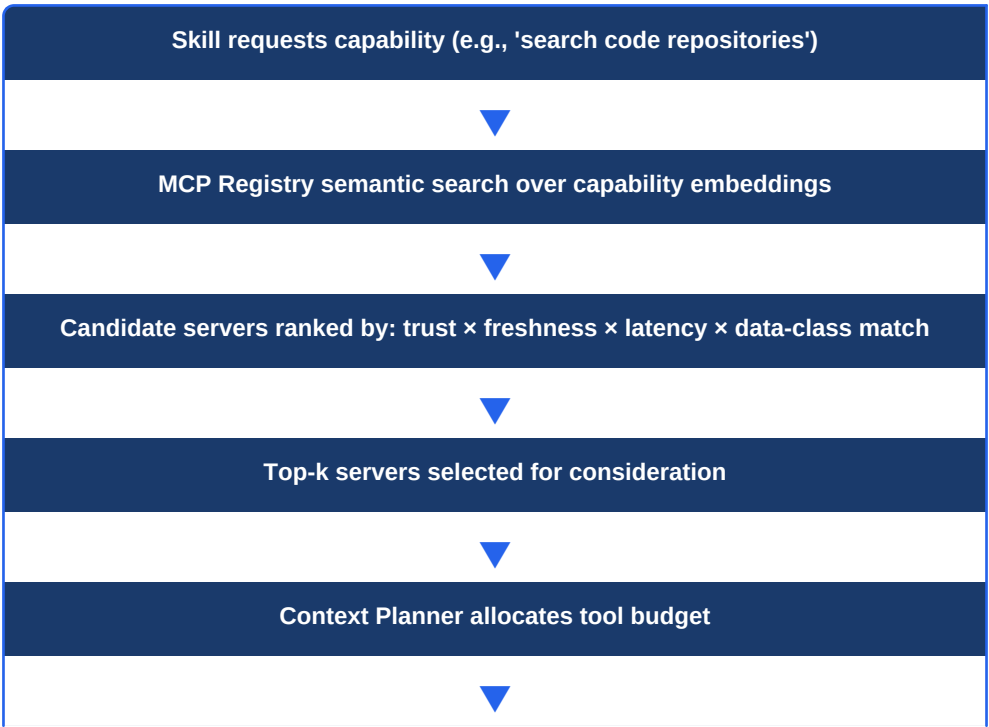
8.1 MCP Capability Registry

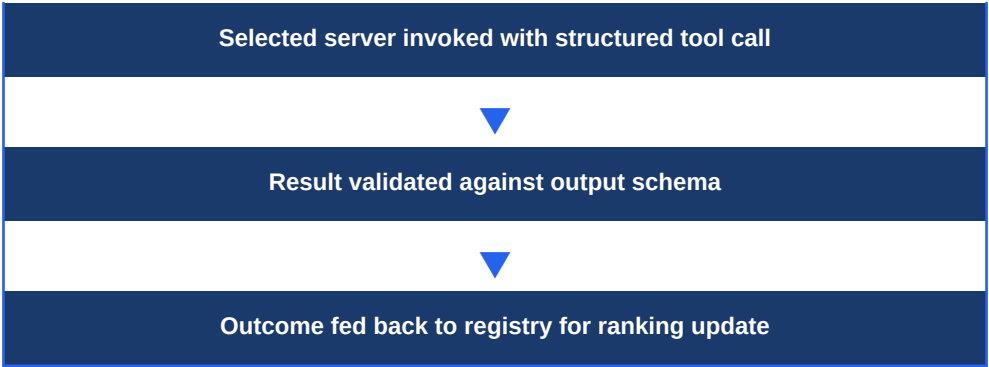
Each registered MCP Server carries a structured capability manifest:

- **server_id** — globally unique identifier.
- **capabilities[]** — semantic capability descriptions (not just tool names).
- **tools[]** — full tool schema with input/output types and rate limits.
- **auth_model** — OAuth2, API key, mTLS, or identity-federated.
- **data_classifications[]** — what data tiers this server may access.
- **trust_level** — governance-assigned trust (Internal / Vendor / Community).
- **health_endpoint** — live health and latency metrics.
- **sla** — p95 latency, availability SLA, and support contact.

8.2 Dynamic MCP Discovery

Dynamic MCP Discovery and Tool Selection





8.3 Multi-MCP Orchestration

- **Parallel invocation** — independent tool calls across servers executed concurrently.
- **Sequential chaining** — output of one MCP tool piped as input to the next.
- **Fallback cascading** — if primary server fails, automatic failover to next-ranked server.
- **Result fusion** — cross-server results merged with source attribution preserved.
- **Budget enforcement** — total tool-call budget capped per KEP stage.

8.4 MCP Security Boundaries

■ MCP server invocations are bound by the data classification of the invoking Skill. A Skill operating on Confidential data may not invoke an MCP server classified below Confidential. All cross-boundary attempts are blocked by the Governance Engine and logged for security review.

8.5 Tool Governance

Control	Mechanism	Enforcement Point
Access control	Skill-level RBAC + data-class matching	MCP Registry at dispatch time
Rate limiting	Per-server token bucket; burst allowance	MCP Client wrapper
Input validation	JSON Schema validation before dispatch	Skill Composer
Output validation	Schema + hallucination detector post-call	Evaluation Engine
Audit logging	Every tool call logged with KEP reference	Observability Layer
Deprecation	Tool version pinning; migration alerts	MCP Registry subscription

9. Microsoft Agent Ecosystem Integration

Microsoft's agent ecosystem — spanning Azure AI Foundry, Microsoft 365 Copilot, Azure AI Agent Service, and Microsoft Fabric — provides enterprise-grade orchestration, identity, and governance infrastructure. EAKA integrates natively with this ecosystem while remaining vendor-neutral through open standards.

9.1 Integration Architecture

Layer	Microsoft Component	EAKA Integration Point
Identity & Auth	Entra ID (AAD), MSAL	Skill access control via OAuth2 + RBAC tokens
Orchestration	Azure AI Agent Service	EAKA Skill Composer as custom agent plugin
Knowledge	Microsoft Graph, SharePoint	T2/T3 source connectors in Federated Discovery Engine
Collaboration	Microsoft 365 Copilot	EAKA skills surfaced as Copilot extensions
Data Platform	Microsoft Fabric / Purview	Data lineage, classification, and policy enforcement
Model Hosting	Azure OpenAI / AI Foundry	Model endpoints for Skill execution and evaluation
Monitoring	Azure Monitor, Application Insights	EAKA observability telemetry pipeline
DevOps	Azure DevOps, GitHub Actions	Skill CI/CD pipeline integration

9.2 Skill Discovery via Microsoft Ecosystem

- Skills registered in EAKA Skill Registry are exposed as **Microsoft Copilot Plugins** via OpenAPI manifest.
- Azure AI Agent Service discovers EAKA skills through the **Agent Plugin Catalogue**.
- Skill metadata is indexed in **Microsoft Graph** for organisation-wide discoverability.
- Entra ID group membership drives skill-level access control automatically.

9.3 Ecosystem Comparison

Dimension	Microsoft (AI Foundry)	AWS (Bedrock Agents)	Google (Vertex AI)	Open Source (LangGraph)
Orchestration	Agent Service + Semantic Kernel	Bedrock Agent DAGs	Agent Builder	LangGraph / CrewAI
Skill/Tool model	Copilot Plugins + A2A	Action Groups	Extensions	Tools / Custom nodes
Knowledge store	Azure AI Search + Graph	Knowledge Bases (S3)	Vertex Search	Any (pluggable)

Dimension	Microsoft (AI Foundry)	AWS (Bedrock Agents)	Google (Vertex AI)	Open Source (LangGraph)
Identity	Entra ID (enterprise-grade)	IAM Roles	Workspace Identity	None (bring-your-own)
Governance	Purview + AI Foundry guardrails	Bedrock Guardrails	Model Armor	None native
MCP support	Preview (mid-2025+)	Partial (custom)	Partial (via SDK)	Full (open-source)
Openness	Proprietary + open standards	Proprietary	Proprietary	Fully open
EAKA fit	Best for enterprises on M365	Best for AWS-native	Best for GCP-native	Best for custom/hybrid

10. Knowledge Context Engineering

Context Engineering is the discipline of optimally managing what knowledge an agent loads, compresses, summarises, retrieves, forgets, and refreshes within a bounded context window. Poor context management is the primary driver of hallucination and stale-knowledge errors in enterprise AI systems.

10.1 Context Budget Framework

Context Zone	Budget Allocation	Content	Eviction Policy
System Context	5–10%	Agent identity, active skill specs, governance rules	Pinned — never evicted
Goal & KEP	10–15%	User goal, Knowledge Execution Plan, validation rules	Pinned until task complete
Active Skills	20–30%	Executing skill prompts, retrieval strategies	LRU within skill budget
Retrieved Knowledge	30–40%	Chunked knowledge artefacts with provenance	Relevance + trust decay
Tool Results	10–15%	MCP tool outputs, structured data	Summarised after use
Working Memory	5–10%	Intermediate reasoning steps, entity state	Checkpoint + compress

10.2 Context Lifecycle Operations

- **Load** — inject knowledge chunk into context, tagged with source, trust score, and TTL.
- **Compress** — summarise verbose tool output or long documents to essential claims.
- **Forget** — evict low-relevance, low-trust, or expired chunks on LRU + trust-weighted policy.
- **Refresh** — re-retrieve a chunk whose TTL has expired or whose source has been updated.
- **Checkpoint** — serialise working memory for long-horizon tasks that exceed one context window.
- **Summarise** — distil multi-turn reasoning into compact facts for handoff to next agent.

10.3 When to Retrieve vs. Use Cached Knowledge

■ Retrieve freshly when: (a) source freshness TTL has expired, (b) task is safety-critical or compliance-driven, (c) knowledge was flagged as recently updated in the EKG. Use cached knowledge when: (a) TTL is valid, (b) task is low-risk exploratory, (c) retrieval latency would exceed SLA budget.

10.4 Hallucination Reduction Through Context Engineering

- Ground every factual claim to a retrieved chunk with provenance (not model parametric memory).
- Enforce a 'no-claim-without-citation' rule in skill output schemas.

- Use trust-score thresholds: only chunks with trust ≥ 65 are used for safety-critical claims.
- Cross-validate critical facts across ≥ 2 independent sources before including in response.

11. Agent Reliability

Reliability in enterprise AI is not simply about model accuracy — it encompasses knowledge currency, retrieval fidelity, tool robustness, and architectural consistency. EAKA defines a multi-dimensional reliability framework.

11.1 Failure Taxonomy

Failure Type	Root Cause	EAKA Mitigation
Knowledge hallucination	Model parametric memory overriding retrieved facts	Canonical mandatory output schema; grounding checks
Stale documentation	Index not updated after source change	Change-event-driven re-indexing; TTL freshness gates
Incorrect SDK version	Version-agnostic retrieval; no version pinning	SDK version nodes in EKG; version-aware retrieval
Wrong implementation pattern	Low-trust source ranked above authoritative	Trust score hierarchy enforced in retrieval fusion
Conflicting ADRs	Multiple ADRs without supersedes relationships	Conflict detection; governance resolution protocol
Retrieval failure	Source system downtime; index corruption	Multi-source redundancy; degraded-mode fallback policy
Tool failure	MCP server timeout or schema mismatch	Fallback server cascading; retry with exponential backoff
Composition error	Skill dependency cycle or budget exhaustion	Self cycle detection; budget enforcement in Skill Composer

11.2 Reliability Metrics

- **Knowledge Grounding Rate (KGR)** — % of factual claims traceable to a retrieved chunk.
- **Source Freshness Score (SFS)** — weighted average freshness of all chunks used in a response.
- **Tool Success Rate (TSR)** — % of MCP tool calls that succeed within SLA.
- **Skill Composition Accuracy (SCA)** — % of compositions that satisfy goal requirements.
- **Conflict Resolution Time (CRT)** — average time to resolve a detected knowledge conflict.
- **Hallucination Detection Rate (HDR)** — % of hallucinations caught by the Evaluation Engine.

11.3 Reliability Measurement Architecture

- **Online evaluation** — real-time fact-checking via Evaluation Engine on every KEP execution.
- **Offline evaluation** — scheduled batch runs of Skill evaluation suites against golden datasets.
- **Adversarial testing** — red-team probing of skills with known-bad or conflicting inputs.
- **User feedback loop** — thumbs-up/down and correction signals fed back to Trust Engine.
- **A/B skill versioning** — canary deployments of new skill versions with statistical comparison.

12. Enterprise Reference Architecture

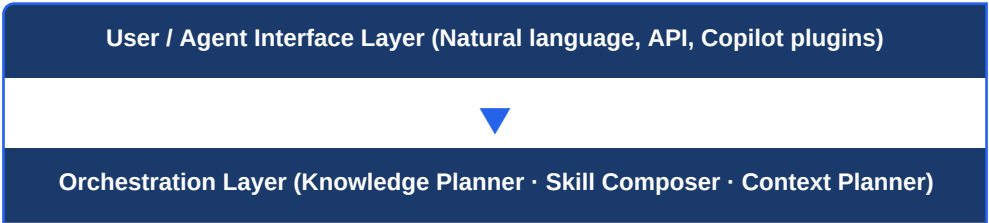
The EAKA Reference Architecture defines a layered, loosely coupled platform with clear separation between knowledge representation, skill execution, governance, and observability. All components communicate through well-defined APIs, enabling vendor-neutral deployment.

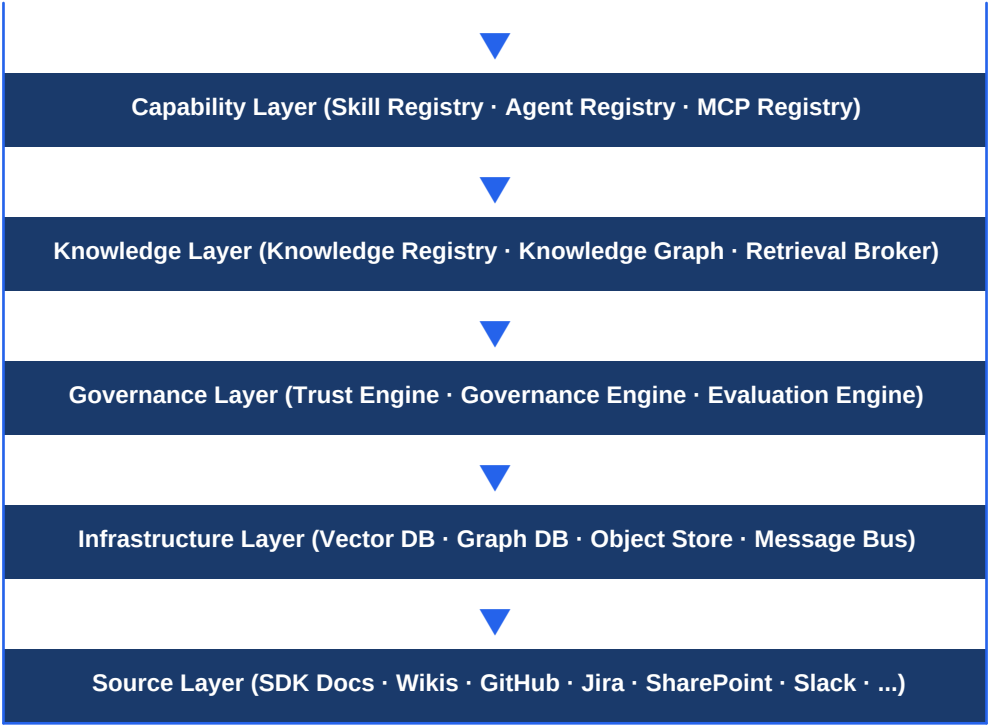
12.1 Platform Component Inventory

Component	Responsibility	Key Interfaces
Knowledge Registry	Stores and indexes all knowledge artefacts with proper metadata	Insert API, Search API, Change Event Stream
Skill Registry	Governs skill lifecycle: publish, version, discover, retire	Skill CRUD API, Discovery API, Subscription Events
Agent Registry	Tracks agent identities, capabilities, and active tasks	Agent Register API, Health API, Task Lifecycle API
MCP Registry	Manages MCP server catalogue, capability index and health	Server Register API, Tool Discovery API, Health Stream
Context Planner	Allocates context budgets and manages windowing	Budget Allocation API, Eviction Policy Engine
Knowledge Planner	Constructs Knowledge Execution Plans from user goals	Goal Parse API, KEP Generate API, KEP Archive
Skill Composer	Assembles skill DAGs and manages execution orchestration	Compose API, Execute API, DAG Visualisation API
Retrieval Broker	Routes retrieval queries to correct sources with fusion	Query API, Source Selector, Fusion Engine
Trust Engine	Computes and maintains trust scores for all artefacts	Score API, Decay Jobs, Feedback Ingestion API
Governance Engine	Enforces policies, approvals, conflicts, and compliance	Policy API, Approval Workflow, Audit Log API
Evaluation Engine	Runs evaluation suites and hallucination detection	Eval Run API, Metric Store, Alert Dispatch
Knowledge Graph	Persists and queries the Enterprise Knowledge Graph	Graph Query API (SPARQL/Cypher), Change Stream
Observability	Centralised telemetry, tracing, and alerting	OpenTelemetry collector, Dashboard API, Alert API
Feedback Loop	Collects user and agent signals to improve knowledge quality	Feedback Ingest API, Signal Processing, Trust Update

12.2 Layered Architecture

EAKA Layered Reference Architecture





12.3 Technology Stack Recommendations

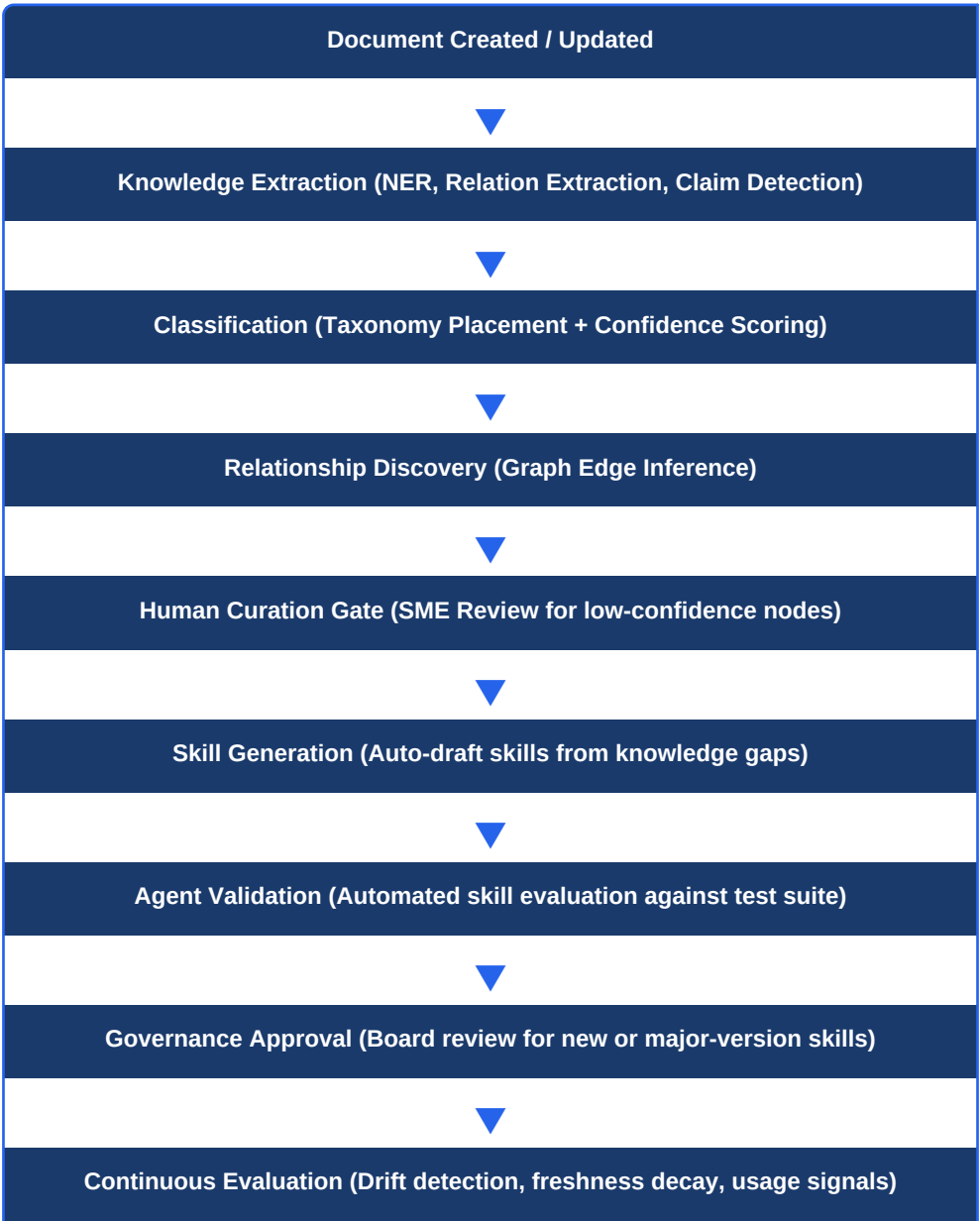
Layer	Open-Source Options	Cloud-Managed Options
Vector Store	Qdrant, Weaviate, Milvus	Azure AI Search, AWS OpenSearch, Google Matching Engine
Graph Database	Neo4j Community, Apache Age (Postgres)	Neo4j Aura, Amazon Neptune, TigerGraph Cloud
Message Bus	Apache Kafka, NATS.io	Azure Service Bus, AWS EventBridge, GCP Pub/Sub
Observability	OpenTelemetry + Grafana + Tempo	Azure Monitor, AWS X-Ray, Google Cloud Trace
MCP Runtime	Open-source MCP SDK (Anthropic)	Vendor-integrated (Azure AI Foundry)
Workflow Engine	Temporal.io, Apache Airflow	AWS Step Functions, Azure Durable Functions
API Gateway	Kong, Envoy, Traefik	Azure APIM, AWS API Gateway, GCP API Gateway

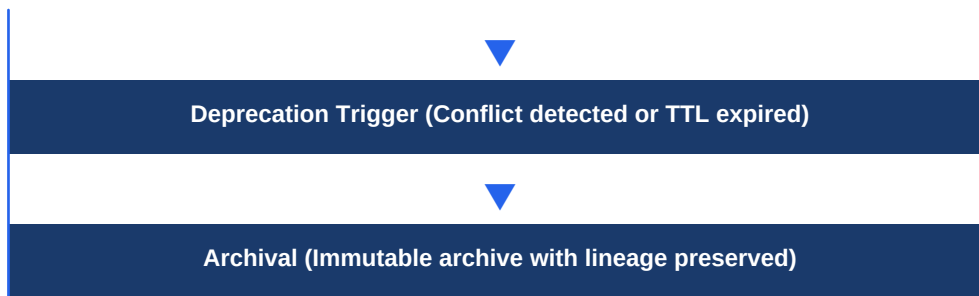
13. AI-Assisted Knowledge Lifecycle

Enterprise knowledge must not merely be indexed but continuously curated, validated, and evolved. EAKA's Knowledge Lifecycle Engine uses AI to automate the pipeline from raw document creation to governed skill generation — with human oversight at key gates.

13.1 Lifecycle Pipeline

AI-Assisted Knowledge Lifecycle Pipeline





13.2 AI-Generated Skill Drafts

When new documentation is ingested, the Lifecycle Engine analyses knowledge gaps in the Skill Registry and automatically drafts new skills:

- Identify concepts in new document not covered by any active skill.
- Draft skill metadata: purpose, scope, required knowledge, retrieval strategy.
- Generate initial prompt templates using in-context examples from similar skills.
- Create skeleton evaluation suite with auto-generated test cases.
- Submit draft to Skill Registry with status 'Draft — Pending SME Review'.

13.3 Drift Detection

- **Semantic drift** — embedding distance between current skill prompts and updated source docs exceeds threshold.
- **Version drift** — SDK or API version referenced in skill no longer matches latest released version.
- **Policy drift** — governing policy has been updated since skill last validated against it.
- **Usage drift** — skill success rate drops below reliability threshold in production metrics.

■ Detected drift triggers an automated notification to the skill owner with a diff between current skill specification and the updated source material. Owners have 30 days to publish an update before the skill is flagged as 'At Risk'.

14. Maturity Model & Roadmap

Enterprises should not attempt to implement the full EAKA platform in a single programme. The EAKA Maturity Model defines five progressive levels, each delivering measurable value while building towards the complete architecture.

14.1 Enterprise AI Knowledge Maturity Model

Level	Name	Capabilities	Key Deliverable
L1	Ad-hoc RAG	Single vector index; no governance; manual tool control	Working AI assistant with document search
L2	Governed Retrieval	Multi-source connectors; provenance tracking; trust scoring	Knowledge Registry + Source Tiers
L3	Skill-Enabled	Skill Registry; basic skill composition; versioned skills	Enterprise Skills Platform v1
L4	Knowledge-Planned	KEP engine; EKG; dynamic MCP; context engineering	Knowledge Planner + EKG live
L5	Autonomous Evolution	AI-assisted lifecycle; drift detection; self-improving skills	Full EAKA Platform — continuous improvement

14.2 Implementation Roadmap

Phase	Duration	Milestones	Success Metrics
Phase 0 Foundation	0–3 months	Source connector pilot (3 systems); vector index; basic trust scoring	2 source systems indexed; trust scores live
Phase 1 Skill MVP	3–6 months	Skill Registry; 10 pilot skills; governance workflow; eval suite	80% skill eval pass rate
Phase 2 KEP Engine	6–9 months	Knowledge Planner; EKG beta; dynamic MCP discovery; context budgeting	KEP, context budgeting ≥30%
Phase 3 Composition	9–14 months	Dynamic Skill Composer; multi-MCP orchestration; MCP self-composition	15 self-composition working
Phase 4 Lifecycle	14–20 months	AI-assisted lifecycle; drift detection; auto skill drafting	Skill currency ≥90% across registry
Phase 5 Scale	20–28 months	Enterprise-wide rollout; open platform publication; continuous improvement	10,000 skills; 50 MCP servers governed

14.3 Gap Analysis — Unsolved Research Problems

- **Cross-agent knowledge negotiation** — no standard protocol for agents to agree on conflicting knowledge in multi-agent systems.
- **Real-time graph consistency** — maintaining EKG consistency under concurrent high-frequency updates without locking.
- **Skill transfer learning** — automatically adapting skills from one enterprise domain to another with minimal re-curation.
- **Context compression quality** — lossless compression of complex technical knowledge within tight token budgets.
- **Trust score calibration** — ground-truth labelling of trust scores at enterprise scale remains labour-intensive.
- **Adversarial knowledge injection** — detecting and mitigating attempts to poison enterprise knowledge sources.
- **Cognitive load of governance** — SME review bottlenecks at scale; automation of approval without sacrificing rigour.

Decision Matrix — Platform Comparison

The following matrix compares leading enterprise AI platforms against the fourteen EAKA capability dimensions. Scores are based on publicly available documentation and analyst assessments as of the publication date.

Capability	EAKA (this work)	Azure AI Foundry	AWS Bedrock Agents	Google Vertex AI	LangChain/ LangGraph	Cohere Compass
Knowledge Discovery	★★★★★	★★★★■	★★★██	★★★★■	★★★██	★★★★■
Knowledge Classification	★★★★★	★★★██	★★███	★★★██	★★███	★★★★■
Knowledge Planning (KEP)	★★★★★	★★███	★★███	★★███	★★★██	★███
Skill Architecture	★★★★★	★★★★■	★★★██	★★★██	★★★██	★★★██
Dynamic Composition	★★★★★	★★★██	★★███	★★★██	★★★★■	★★███
Governance	★★★★★	★★★★★	★★★★■	★★★██	★★███	★★★██
Knowledge Graph	★★★★★	★★★██	★★███	★★★██	★★███	★★★██
MCP Integration	★★★★★	★★★██	★★███	★★███	★★★★★	★███
Context Engineering	★★★★★	★★★██	★★★██	★★★██	★★★██	★★███
AI Knowledge Lifecycle	★★★★★	★★★██	★★███	★★★██	★★███	★★★★■
Reliability Framework	★★★★★	★★★★■	★★★★■	★★★██	★★███	★★★██
Vendor Neutrality	★★★★★	★★███	★★███	★★███	★★★★★	★★███

★★★★★ = Full support ★★★★■ = Strong ★★★██ = Partial ★★███ = Limited ★███ = None